



Module 4 - Lecture Notes

DevOps, CI/CD, Documentation & Team Workflows with AI

AI-Assisted Coding

Module snapshot

Module 4 moves from feature-building to the engineering layer around the existing Task Tracker. You will use Claude Code in the terminal to prepare the repository for sharing and maintenance: project memory, CI, Docker, documentation, AI-assisted review, and a technical decision note.

The central habit is the same loop from earlier modules: ask -> inspect -> run -> test -> refine. The difference is scope: Claude Code can affect the whole repo, so every generated artifact needs evidence before you accept it.

How to use these notes

Read these notes alongside the Module 4 videos. They are not a transcript. They are a student-facing guide to the concepts, examples, verification steps, and deliverables taught in the module.

Use the checkpoints as pause points during the demos. Do not move on just because an AI-generated file looks plausible; move on when you have inspected it and produced evidence that it works.

Learning outcomes

1. Explain why CI files, Dockerfiles, and documentation can look correct while quietly being wrong.
2. Apply the three Module 4 rules: read the diff before approving, verify before moving on, and never let AI approve work for you.
3. Choose the right AI surface for the task: Copilot for line-level help, Cursor for IDE/file-level work, and Claude Code for repo-level terminal work.
4. Set up Claude Code in the correct Task Tracker repository and use `/status`, `/init`, plan mode, permissions, slash commands, and `@file` references safely.
5. Create and hand-correct CLAUDE.md with the project stack, run/test commands, business rules, UI states, CORS expectations, and do-not rules.

6. Generate a GitHub Actions CI workflow with Claude Code, inspect it for dangerous shortcuts, and prove it works with green -> red -> green evidence.
7. Generate a multi-stage Dockerfile and .dockerignore, then verify the running container with /health and a non-root user check.
8. Generate docstrings and README updates with AI, then verify the claims against code, FastAPI /docs, schemas, and actual status codes.
9. Use AI-assisted code review as triage by classifying every comment as Useful, Noise, or Wrong.
10. Write a technical decision note in your own voice, using Claude for structure but owning the trade-offs and open questions.

Module outputs

Output	What it should contain	Why it matters
Claude Code setup	Claude Code installed/authenticated, correct repo opened, CLAUDE.md committed, and two Claude answers verified against actual files.	Gives the terminal agent real project memory before it edits or runs anything important.
CI workflow evidence	.github/workflows/ci.yml plus evidence of green -> intentional red -> restored green runs.	A green check only matters if a broken test turns it red.
Docker artifacts	Multi-stage Dockerfile, .dockerignore, local build/run evidence, /health check, and non-root whoami check.	Shows the app can run consistently and more safely outside your local environment.
Docker security log	A short log confirming non-root user, slim runtime base, and no baked secrets.	Turns “it builds” into “it is closer to shippable.”
Verified documentation	Docstrings and README updates plus a claim-vs-reality log with inaccuracies caught and fixed.	Prevents fluent false documentation from becoming official.
Annotated review log	Every Claude review comment categorized as Useful, Noise, or Wrong, with a short justification.	Builds judgment about where AI review helps and where it distracts.
Technical note and reflection	docs/decisions/<short-name>.md linked from README, rewritten in your own voice, plus a tool-fit reflection comparing Copilot, Cursor, and Claude Code.	Preserves the “why” behind decisions and turns tool use into engineering judgment.

Part 1 - Module 4 Mindset and Build Sequence

Key takeaway

We are not adding new product features. We are wrapping the existing Task Tracker in the engineering artifacts it needs to be shared, maintained, reviewed, and eventually shipped.

Module 4 changes the kind of work you are doing. Earlier modules focused on requirements, scaffolded code, tests, and feature implementation. This module focuses on the delivery layer around the app: CI, containerization, documentation, code review, and technical decisions.

This layer can be deceptive. A CI workflow can show a green check while skipping failures. A Dockerfile can build but run as root. Documentation can sound confident while describing behavior the code does not have. That is why the module repeatedly returns to verification.

Three strict rules	What it means in Module 4
Read the diff before approving.	Claude Code may edit multiple files in one turn. The broader the scope, the higher the cost of careless approval.
Verify before moving on.	Generated artifacts are not done just because they exist.
AI never approves.	Claude can propose commands, diffs, reviews, and documentation, but it does not own the result.

One repo, six connected parts

Everything happens in the same Task Tracker repository. The sequence is intentionally continuous: project memory first, then CI, then Docker, then documentation, then review, then a technical note. Each step gives you an artifact that later steps can inspect or explain.

Part	Focus	Main evidence
4.1	Set up Claude Code and create reliable project memory.	CLAUDE.md and two verified interactions.
4.2	Create CI and prove it catches broken tests.	green -> red -> green CI evidence.
4.3	Containerize the app safely.	Docker build/run, /health, non-root whoami.
4.4	Generate and verify documentation.	docstrings, README, claim-vs-reality log.
4.5	Use AI-assisted review as triage.	Useful / Noise / Wrong review log.
4.6	Write a technical decision note.	Decision note linked from README and rewritten in your voice.

Review question

Which artifact in this module would be most dangerous if it looked correct but was wrong: CI, Docker, documentation, review, or the technical note? Why?

Part 2 - Choosing the Right AI Surface

Key takeaway

The module compares tools by scope rather than ranking them. The broader the tool scope, the stricter your verification habit must be.

AI surface	Best fit	Not ideal for	What to verify
Copilot / autocomplete	Line-level help, small functions, familiar patterns, filling in repetitive code.	Work that crosses files or requires running commands.	Inspect the exact code it inserted.
Cursor / IDE chat	File-level or project edits inside the editor, sidebar chat, explanations, diffs over visible code.	Terminal-heavy work such as CI, Docker, git, and test loops.	Review returned diffs and run the project.
Claude Code / terminal agent	Repo-level tasks: reading files, planning, editing multiple files, running pytest, Docker, git, or gh.	Tiny one-line edits where a full terminal agent adds unnecessary risk.	Use CLAUDE.md, plan mode, permissions, diffs, commands, and evidence.

Core loop

The loop stays the same across tools: ask -> inspect -> run -> test -> refine. The scope changes, so the cost of skipping a step increases.

Part 3 - Set Up Claude Code in the Terminal

Key takeaway

A terminal agent without project memory behaves like a smart outsider. Before asking Claude Code to make important changes, give it accurate repo-specific context and verify that it understands the project.

Part 4.1 is about setup, but installation is not the main point. The real goal is to prepare Claude Code so it understands the actual Task Tracker repository rather than a generic FastAPI project.

What makes Claude Code different

Claude Code works from the terminal. It can read the repository, plan multiple steps, edit files, and run shell commands. That makes it useful for CI, Docker, documentation, and review tasks. It also makes it risky: one approval can affect several files or run commands that change the repo.

Concept	Meaning	How it is used in the module
CLAUDE.md	Project memory for the agent.	Stores stack, commands, architecture, business rules, UI states, CORS, and do-not rules.
Plan mode	A safety mode where Claude proposes a plan before editing.	Use it before larger changes so you can inspect the intended steps first.
Permissions	Approval gates before edits and shell commands.	Reads are generally lower risk; writes and commands require conscious approval.
@file references	A way to pull specific files into context.	Use @app/main.py or @app/models.py when asking questions that must be verified against code.
Slash commands	Built-in Claude Code actions such as /status and /init.	Use /status to confirm session context and /init to draft CLAUDE.md.

CLAUDE.md: project memory, not decoration

Run /init to generate a draft, but do not commit the draft as-is. Your job is to correct it by hand so it reflects this repository.

CLAUDE.md section	What to include for Task Tracker
Stack	Python 3.11, FastAPI, Pydantic v2, Uvicorn, pytest, httpx, and the vanilla JavaScript frontend.
Run and test commands	Use the exact commands from your repo, such as <code>uvicorn app.main:app --reload --port 8000</code> and <code>pytest -v</code> .
Architecture	Where the FastAPI app, models, frontend, tests, and supporting files live.
Business rules	Task statuses, allowed transitions, invalid update behavior, missing task behavior, and any rules that should not be guessed.
UI states	Task columns and frontend states that matter to app behavior.
CORS and local dev notes	What origins or ports are expected in the local setup.
Do-not rules	Do not add authentication, a database, unrelated UI changes, deployment steps, or secrets unless explicitly approved.

Hands-on flow

1. Install and authenticate Claude Code using the course environment instructions.
2. Open the correct Task Tracker repository. Do not start in the parent folder or a nested subfolder.
3. Run `/status` and, if needed, `/model` to confirm your session context.
4. Run `/init` to generate `CLAUDE.md`.
5. Read the generated `CLAUDE.md` and correct it by hand.
6. Use plan mode for a small proposed change, such as asking for a plan for a `/version` endpoint without editing files yet.
7. Ask two simple project questions and verify the answers against `app/main.py` and `app/models.py` before moving on.

Review question

Does `CLAUDE.md` describe this specific Task Tracker project, or could it describe any generic FastAPI app?

Part 4 - Create the CI Pipeline

Key takeaway

A green check only matters if failure turns it red. CI is not proved by one passing run; it is proved when a deliberate test failure makes the workflow fail.

Part 4.2 is where the project starts protecting itself. Local tests help only when someone remembers to run them. Continuous Integration runs the test suite automatically on push and pull request so broken code is harder to merge unnoticed.

Expected workflow shape

Use Claude Code to generate `.github/workflows/ci.yml`, but inspect the YAML line by line. The module expects a simple test workflow, not deployment automation.

Workflow area	Expected behavior
Location	<code>.github/workflows/ci.yml</code> at the repository root.
Triggers	Run on push to branches and pull request to main.

Workflow area	Expected behavior
Job	One job named test running on ubuntu-latest.
Python setup	Use the exact project Python version from CLAUDE.md or the repo configuration. In the demo context, this is Python 3.11.
Dependencies	Install the project requirements, including test dependencies such as pytest and httpx when the test suite needs them.
Test command	Run pytest in a way that returns a non-zero exit code when tests fail.

Reject these immediately

Red flag	Why it is dangerous
continue-on-error	Allows a failing test step to appear acceptable.
true	Turns a failing command into a successful shell command.
--exit-zero or similar flags	Can force a tool to exit successfully even when problems exist.
latest Python or vague version	May test on a different version than students run locally.
Missing push or PR trigger	The workflow may not run when it actually matters.
Deployment steps	This module is not deploying the app; deployment adds risk and scope that the lecture did not ask for.

The green-red-green proof

Step	What to do	What proves it worked
Green	Push the workflow and confirm the normal test suite passes.	GitHub Actions shows a successful run.
Red	Deliberately break one test on a branch and push.	The CI run fails and the log shows the same kind of test failure you would expect locally.
Green again	Revert the deliberate break and push again.	CI returns to green without changing the workflow to hide the failure.

Demo troubleshooting note

The recording shows realistic friction: the workflow can land in the wrong folder, or CI can fail because test dependencies such as pytest or httpx are missing from requirements. Fix the repo truthfully; do not hide the failure.

Part 5 - Containerize the Task Tracker

Key takeaway

A Dockerfile can build successfully and still be poor engineering. The goal is not only “it builds”; the goal is a smaller, safer runtime image that runs the actual app without secrets.

Part 4.3 turns the working Task Tracker app into something that can run consistently in another environment. Claude Code can draft the Dockerfile and .dockerignore, but the human inspection is the important part.

What the Dockerfile should do

Requirement	What to check
Multi-stage build	Use a builder stage for dependency preparation and a runtime stage that copies only what is needed to run.
Explicit slim Python base	Use an explicit slim runtime base such as python:3.11-slim. Avoid floating latest tags.
Non-root user	Create an app user, set ownership, and make sure USER app appears before CMD.
No baked secrets	Do not copy .env, credentials, tokens, or private files into the image.
Production-ish command	Run uvicorn app.main:app with --host 0.0.0.0 --port 8000 and no reload flag.
Health check	Verify the running container responds at /health.

.dockerignore expectations

The .dockerignore file is part of the security story. It should exclude files that do not belong in the image, including .env, .git, venv or .venv, tests when not needed at runtime, caches, build artifacts, and local machine files.

Local verification commands

```
docker build -t task-tracker:dev .
docker run --rm -d -p 8000:8000 --name tt-dev task-tracker:dev
curl -i http://localhost:8000/health
docker exec tt-dev whoami
docker stop tt-dev
```

The expected security check is that whoami returns app, not root. Also record the image size and investigate if it is unexpectedly large, but do not treat size alone as proof of correctness.

Three-line Docker security log	Example entry
Non-root user	docker exec tt-dev whoami -> app
Slim runtime base	Runtime stage uses an explicit slim Python base.
No baked secrets	.dockerignore excludes .env, .git, virtual environments, caches, and build artifacts.

Part 6 - Generate and Verify Documentation

Key takeaway

The most dangerous documentation is the documentation that sounds right. Fluent prose is not evidence; documentation must be checked against the actual code and running API.

Part 4.4 uses AI for documentation, but the real skill is claim verification. If a docstring says a function validates something and the function does not actually validate it, the documentation has turned a missing behavior into an official claim.

What to ask Claude to generate

Documentation artifact	Expected content
Docstrings	Google-style docstrings for every public function and route handler in app/: summary, args, returns with type/shape, raises with actual exceptions, and examples for route handlers.
README.md	Setup, run, test, container, CI, and project conventions. Commands must match the actual repository.

Documentation artifact	Expected content
No behavior changes	Documentation work should not change app logic unless you explicitly decide to make a code fix after verification.

Verification checklist

1. Inspect the diff first. Confirm Claude changed documentation only, not application logic.
2. Pick at least three docstrings and compare each one to the actual function body.
3. Check status transition documentation against the real status rules in the code.
4. Run the app and open /docs to compare schemas, status codes, and route descriptions.
5. Check examples such as POST returning 201 if the implementation returns 201, DELETE returning 204 if the implementation returns no content, and invalid input returning 422 when FastAPI/Pydantic validation applies.
6. Follow README commands from a clean mental model: would a new teammate be able to run the app with only these instructions?

Claim vs. reality log

AI claimed	Code actually does	Resolution
POST /tasks returns 200.	The route returns 201 Created.	Update the docstring/README to say 201.
DELETE returns a JSON object.	The route returns 204 No Content.	Remove the invented response body from the docs.
Status can move freely between all states.	The model or service enforces specific allowed transitions.	Rewrite the status-rule explanation to match the actual code.

The example rows show the kind of mismatch to look for. Your own log should record the real inaccuracies you catch in your repository.

Part 7 - AI-Assisted Code Review

Key takeaway

AI review is a triage exercise, not a trust exercise. The useful skill is categorizing comments and deciding what deserves action.

Part 4.5 asks you to review a real diff. Good choices include a Module 3 frontend refactor, new tests, the Dockerfile addition, .dockerignore changes, or documentation changes. A toy diff will not teach much.

Review workflow

1. Choose a real branch diff against main, or choose a specific commit if the work is already merged.
2. Ask Claude Code to review the diff. You may use /review if available or a review prompt that tells Claude to focus on correctness, tests, security, and documentation drift.
3. Copy each Claude comment into a review log.
4. Assign every comment to Useful, Noise, or Wrong. Do not merge Noise and Wrong; they have different costs.
5. Do your own human review of the same diff and compare what you caught versus what Claude caught.
6. Write one personal AI-review rule, such as: "I will use Claude review for broad first-pass coverage, but I will only act after verifying the comment in the code."

Bucket	Definition	Action
Useful	A real bug, risk, missing test, misleading doc, or maintainability issue.	Verify it, then fix or create a follow-up task.
Noise	Technically fine but not worth acting on, such as a style preference outside team conventions.	Skip it and record why.
Wrong	Misreads the code, invents a problem, or ignores context in another file.	Do not fix the imaginary issue. Record the misunderstanding.

Wrong deserves its own bucket. Noise wastes a glance. Wrong can waste hours or cause you to "fix" an imaginary bug and introduce a real one.

Part 8 - Write a Technical Note

Key takeaway

Claude drafts. You own the voice.

The module ends with a technical note because code tells us what exists, but not always why it exists. A future teammate may see the status rules, CI choices, or Docker design and wonder why the project uses them. A short note captures the reasoning before it disappears.

Good technical note topics

- Architecture choice for the Task Tracker.
- Status transition rules and why they are enforced this way.
- Refactor approach from Module 3.
- DevOps setup decisions such as CI scope or Docker design.
- The example topic shown in the lecture: In-Memory Dict as the Task Storage Layer.

Required structure

Section	Question it answers
Context	What situation or problem led to the decision?
Decision	What did we decide? State it clearly.
Alternatives Considered	What other options were possible?
Trade-offs	What did the chosen option make easier, and what did it make worse?
Consequences	What changes for the codebase or team because of this choice?
Open Questions	What remains unresolved or worth revisiting later?

Division of labor with Claude

Claude can help with...	You must own...
Reading evidence files such as app/main.py, app/models.py, Dockerfile, .github/workflows/ci.yml, and README.md.	Choosing which evidence matters and rejecting generic claims.
Drafting the six headings and a first-pass summary.	Rewriting the prose so it sounds like your reasoning, not a model-generated summary.
Listing possible alternatives.	Explaining the real trade-offs and constraints from this project.
Suggesting open questions.	Writing open questions from scratch based on what you would actually revisit.

```
# Save the note here
docs/decisions/<short-name>.md
```

Required README action

Add a link from README.md to the decision note.

Required reflection sentence

"I would do this differently..."

Checkpoint

Pause after saving the note. Continue only when the README link works and the note sounds like you, especially in Trade-offs and Open Questions.